



UNICAMP

Relatório do Trabalho 4: Medidas e Transformações Geométricas

Relatório desenvolvido a partir da obtenção de medidas de objetos em imagens digitais e aplicação de transformações geométricas de rotação e escala apresentados nas aulas de MC920 Introdução ao Processamento Digital de Imagem

Prof. Dr. Hélio Pedrini

Letícia Silva Brito
240314

1 Introdução	2
2 Materiais	2
Requerimentos	2
3 Medida de Objetos	2
Execução	3
Entrada e Saída	3
Código	3
Tratamento	3
Contorno	4
Cálculo das medidas	5
Histograma	8
4 Transformação geométrica	10
5 Bibliografia	10

1 Introdução

Este trabalho foi desenvolvido com o intuito de obter algumas medidas de objetos identificados em imagens e realizar operações de rotação e mudança de escala por meio de transformações geométricas.

Para facilitar a compreensão do projeto dividiu-se este relatório em duas partes: medição e transformação geométrica.

Os códigos que apresentam tais aplicações, assim como as imagens de entrada, saída e resultados intermediários, podem ser encontrados no [repositório Github](#) do trabalho.

2 Materiais

Esta seção abordará o que é necessário para a execução e organização do código, de modo a tratar, além do código, a organização das entradas e saídas.

Requerimentos

Este projeto foi desenvolvido na linguagem de programação *Python* na versão 3.12.3, utilizando as seguintes bibliotecas (instaladas em um ambiente virtual, através do comando `pip install ...`), que possibilitaram a criação de algoritmos eficazes para trabalhar com imagens digitais:

Tabela 1: bibliotecas utilizadas e suas respectivas versões	
Biblioteca	Versão
OpenCV	4.11.0
NumPy	2.2.4
Matplotlib	3.10.1
Scikit Image	0.25.2

3 Medida de Objetos

Esta seção dedica-se a destrinchar sobre o processo de medição de objetos identificados em uma imagem, algo que está muito relacionado a problemas de classificação.

A fim de organizar os arquivos de entrada, saída e os código de execução colocou-os no diretório ***measurements***.

Execução

Para a execução do programa deve-se escrever no terminal **`python3 main.py <input-file>`**.

Exemplo:

```
python3 main.py objetos1.png
```

Entrada e Saída

As entradas para este programa são provenientes do [diretório](#) do Prof. Hélió e, no diretório do trabalho, se encontram em **`in-images`**.

Como saída há um relatório da imagem e as imagens intermediárias do processo de identificação dos objetos. Os relatórios de cada imagem encontram-se na pasta **`reports`** e as imagens em **`out-images`**.

Código

Dividiu-se o código em dois arquivos: **`main.py`** e **`properties.py`**. O primeiro é responsável pela funcionalidade geral do programa e o segundo contém as funções de mostrar e salvar imagens, calcular os contornos de cada objeto, a excentricidade e a solidez, além de ser capaz de gerar um histograma de 3 *bins*, para os objetos pequenos, médios e grandes.

Tratamento

Iniciou-se tratando a imagem para binarizar-la, através do código abaixo:

```
src = ski.io.imread(f"in-images/{argv[1]}")
mono = ski.color.rgb2gray(src)
mono = cv2.threshold(mono, 0.8, 1, cv2.THRESH_BINARY)[1]
mono = (mono * 255).astype(np.uint8)
```



Imagem 1: *objetos3.png* imagem original colorida



Imagem 2: Imagem 1 binarizada em preto e branco com *threshold* 0.8

Como é possível ver acima, pouco se perde ao transformar a imagem para monocromática. Para fins comparativos, analisar-se-á conjuntamente com *objetos3.png*, *objetos1.png*.

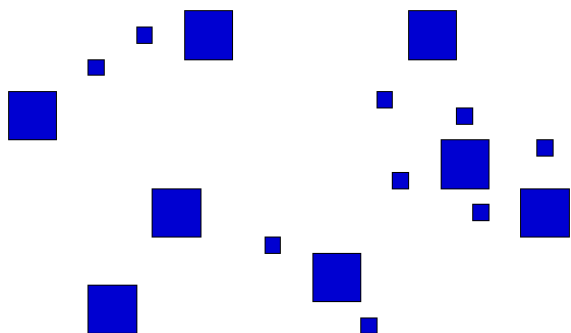


Imagem 3: *objetos1.png* imagem original colorida

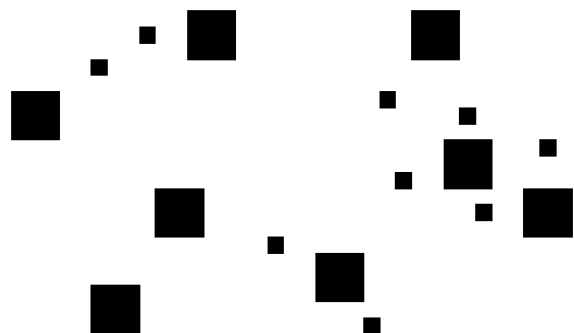


Imagem 4: Imagem 3 binarizada com *threshold* 0.8

Contorno

Com a binarização das imagens, pode-se agora prosseguir à obtenção dos contornos dos objetos que servem de base para o cálculo de algumas propriedades dos objetos em uma figura. Para tal, baseou-se nas documentações da biblioteca OpenCV e por meio do código abaixo, conseguiu-se as seguintes imagens.

```
contours = cv2.findContours(
    src_img,
    cv2.RETR_TREE,
```

```
cv2.CHAIN_APPROX_SIMPLE
)[0]
```

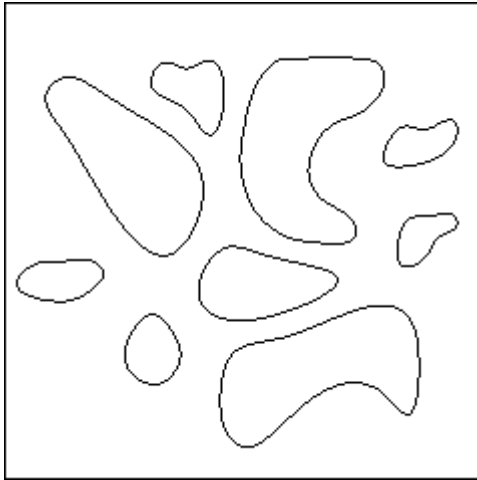


Imagem 5: contornos da Imagem 2

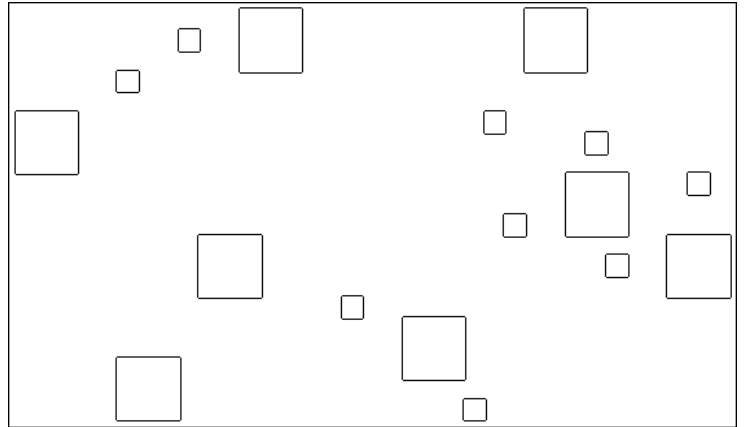


Imagem 6: contornos da Imagem 4

Cálculo das medidas

Com o vetor *contours* extrai-se a quantidade de objetos por meio do método *len()*, cujo resultado deve subtrair 1 para desconsiderar o fundo. Assim, deve-se realizar um *loop* para encontrar a área, o perímetro, a excentricidade e a solidez. Para tal, deve-se obter os momentos do contorno, com $M = cv2.moments(obj)$, onde *obj* é um contorno. Com isso, obtém-se área e perímetro:

```
area = M['m00']
perimeter = cv2.arcLength(obj, True)
```

Para calcular a excentricidade encontrou-se os pontos mais extremos, isto é, aqueles mais à esquerda, mais à direita, mais ao topo e mais abaixo. Calcula-se, assim, as distâncias entre os pontos opostos e, então, a excentricidade, que deve ser a razão do maior eixo pelo menor.

```
extreme_points = [
    tuple(obj[obj[:, :, 0].argmin()][0]),
    tuple(obj[obj[:, :, 0].argmax()][0]),
    tuple(obj[obj[:, :, 1].argmin()][0]),
    tuple(obj[obj[:, :, 1].argmax()][0])
]
```

```

axis_a = np.linalg.norm(
    np.array(extreme_points[0])
    np.array(extreme_points[1])
)

axis_b = np.linalg.norm(
    np.array(extreme_points[2])
    np.array(extreme_points[3])
)

eccentricity = axis_a / axis_b if axis_a > axis_b else
axis_b / axis_a

```

Já para o cálculo da solidez basta calcular a razão de área do contorno e da do "envelope" convexo.

```

area = cv2.contourArea(obj)
hull = cv2.convexHull(obj)
hull_area = cv2.contourArea(hull)
solidity = float(area) / hull_area

```

Com essas informações é possível produzir o relatório dos objetos das imagens:

Relatório de <i>objetos1</i>					
número de regiões: 17					
região 0:	área: 2448	perímetro: 195.656854	excentricidade: 1.400452	solidez: 1.000000	
região 1:	área: 2448	perímetro: 195.656854	excentricidade: 1.400452	solidez: 1.000000	
região 2:	área: 304	perímetro: 67.656854	excentricidade: 1.378279	solidez: 1.000000	
região 3:	área: 304	perímetro: 67.656854	excentricidade: 1.296958	solidez: 1.000000	
região 4:	área: 2399	perímetro: 193.656854	excentricidade: 1.385653	solidez: 1.000000	
região 5:	área: 304	perímetro: 67.656854	excentricidade: 1.378279	solidez: 1.000000	
região 6:	área: 322	perímetro: 69.656854	excentricidade: 1.337955	solidez: 1.000000	
região 7:	área: 2448	perímetro: 195.656854	excentricidade: 1.400452	solidez: 1.000000	
região 8:	área: 322	perímetro: 69.656854	excentricidade: 1.337955	solidez: 1.000000	
região 9:	área: 322	perímetro: 69.656854	excentricidade: 1.337955	solidez: 1.000000	
região 10:	área: 2448	perímetro: 195.656854	excentricidade: 1.371860	solidez: 1.000000	
região 11:	área: 2448	perímetro: 195.656854	excentricidade: 1.371860	solidez: 1.000000	
região 12:	área: 322	perímetro: 69.656854	excentricidade: 1.337955	solidez: 1.000000	
região 13:	área: 304	perímetro: 67.656854	excentricidade: 1.378279	solidez: 1.000000	
região 14:	área: 2399	perímetro: 193.656854	excentricidade: 1.385653	solidez: 1.000000	
região 15:	área: 2448	perímetro: 195.656854	excentricidade: 1.371860	solidez: 1.000000	
região 16:	área: 304	perímetro: 67.656854	excentricidade: 1.296958	solidez: 1.000000	
número de regiões pequenas: 9					
número de regiões médias: 8					
número de regiões grandes: 0					

Relatório de <i>objetos3</i>

número de regiões: 9

região 0: área: 4107 perímetro: 319.421354 excentricidade: 1.113481 solidez: 0.754963
região 1: área: 844 perímetro: 125.639609 excentricidade: 1.023938 solidez: 0.904558
região 2: área: 3690 perímetro: 265.119838 excentricidade: 1.164069 solidez: 0.978264
região 3: área: 584 perímetro: 104.911687 excentricidade: 1.224930 solidez: 0.913928
região 4: área: 478 perímetro: 94.426406 excentricidade: 1.240083 solidez: 0.925460
região 5: área: 1762 perímetro: 179.781745 excentricidade: 1.781643 solidez: 0.971862
região 6: área: 688 perímetro: 108.669047 excentricidade: 2.080398 solidez: 0.972458
região 7: área: 4067 perímetro: 311.078208 excentricidade: 1.139223 solidez: 0.780689
região 8: área: 716 perímetro: 101.982755 excentricidade: 1.243028 solidez: 0.980164

número de regiões pequenas: 5

número de regiões médias: 1

número de regiões grandes: 3

Além dos relatórios, produziu-se imagens com rótulos nos centros dos objetos.

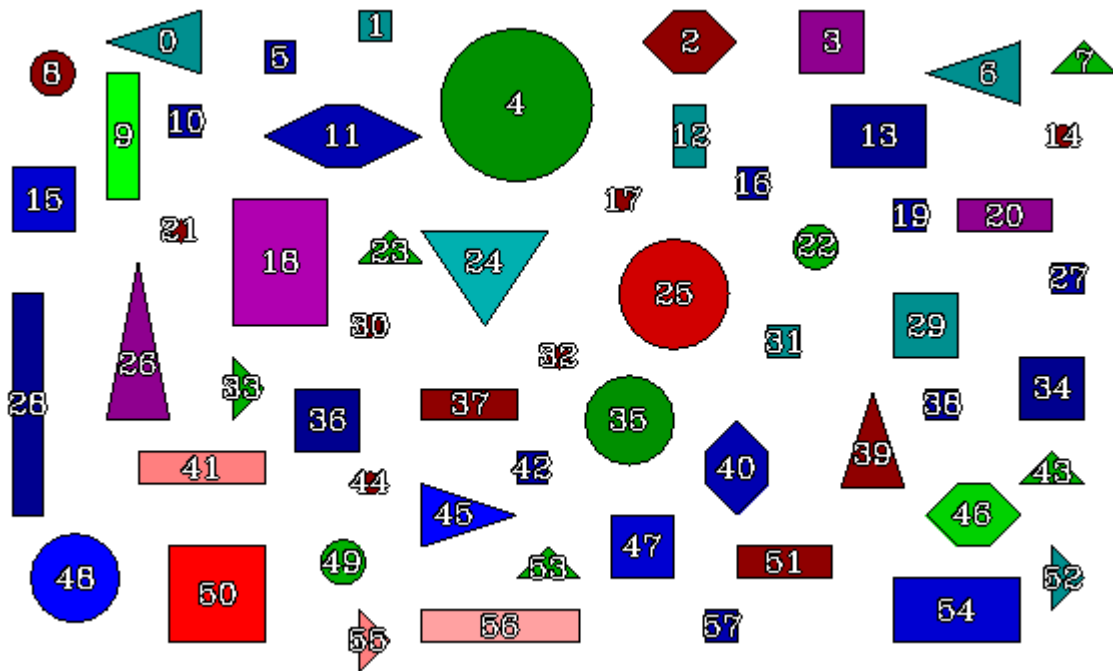


Imagem 7: *objetos1* rotulado.



Imagem 8: *objetos3* rotulado.

Histograma

A partir da obtenção das áreas consegue-se, assim, a determinação do histograma com base na quantidade de objetos pequenos < 1500 , médios $[1500, 3000[$ e grandes ≥ 3000 .

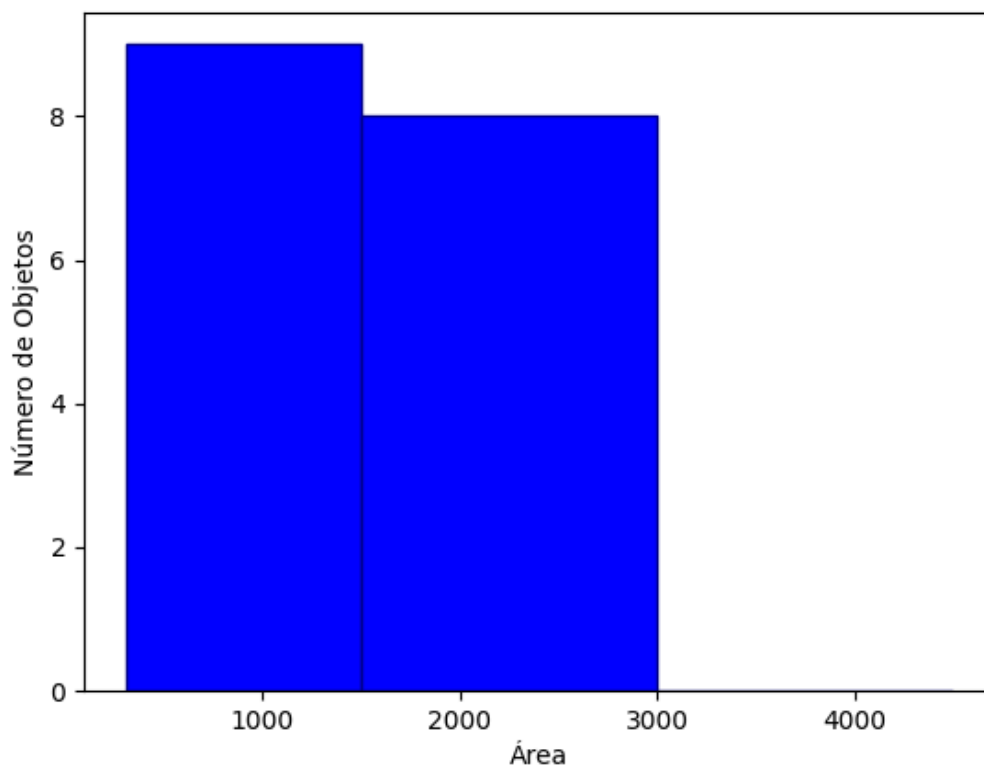


Imagem 9: histograma das áreas de *objetos1*.

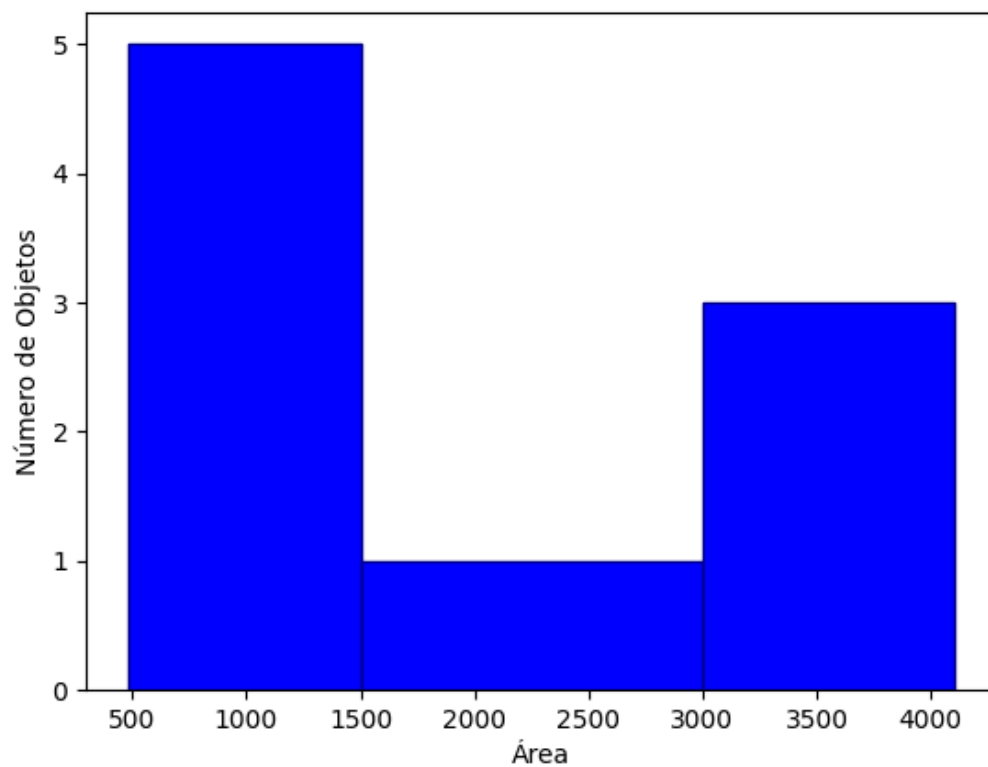


Imagem 10: histograma das áreas de *objetos3*.

4 Transformação geométrica

Esta seção trata sobre os processos de transformação geométrica (rotação e escala) em imagens digitais com base em operações de interpolação.

Esta análise aprofundou-se nas diferentes técnicas de interpolação para as operações de escala e rotação.

A fim de organizar melhor a visualização dos programas, adicionou-se os itens da Transformação Geométrica no diretório **geo-transform/**.

Execução

Para a execução desse programa, seguiu-se o sugerido no enunciado do trabalho, com pequenas alterações:

```
python3 main.py [-a angle]
                 [-s scale]
                 [-d width height]
                 [-m interpolation]
                 [-i input]
```

Os argumentos obrigatórios são apenas **-m** e **-i**.

Entrada e Saída

A entrada do programa consiste em 3 (três) imagens, contidas em um diretório **in-images/**, as quais duas são monocromáticas e uma é colorida. Assim, desenhou o código para que conseguisse trabalhar tanto com bandas monocromáticas quanto com RGB.

Já as saídas foram armazenadas no diretório **out-images/**, de modo que expressam o resultado das operações de **rotação**, **escala** e **redimensionamento**, cujos prefixos adotados foram **rot_**, **esc_** e **res_** (*resize*), respectivamente.

Código

A divisão do código ocorre em **interpolation.py**, **main.py** e **utils.py**. Abaixo, destrinchou-se sobre a **main.py** e **utils.py**, uma vez que **interpolation.py** restringiu-se a implementação dos métodos de interpolação.

utils.py

Este arquivo contém a importação das funções que serão usadas por outros arquivos, sendo estas: *Numpy*, *OpenCv*, *Skimage*, *Matplotlib*, *sys* e *argparse*. Ademais, há funções para mostrar e salvar a imagem na máquina (**show(...)** e **save(...)**). Há, por fim, a função **read()** que é responsável pela leitura

dos argumentos passados pelo terminal, de modo a serem usados pelo programa principal, *main.py*.

main.py

Este trata-se do arquivo principal que chama as interpolações. Mas antes, realiza-se a obtenção de uma matriz de rotação e o correto endereçamento das escalas, de modo que consegue-se obter novos valores dos índices de coluna e linha e, assim, realizar a operação de escala ou rotação.

Desta maneira, confere-se abaixo a rotação da imagem *baboon.png* em um ângulo de 42.0° com diferentes tipos de interpolação.

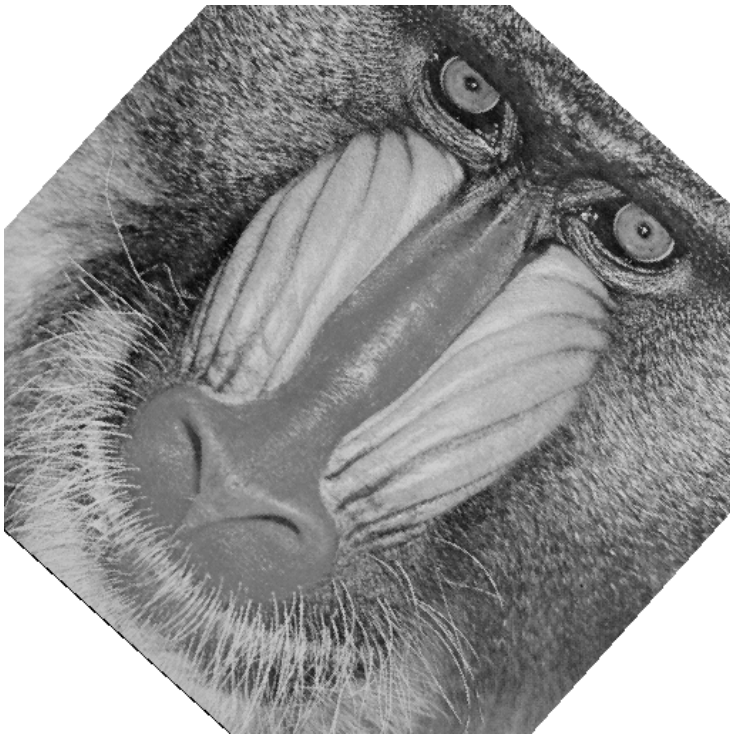


Imagem 1: rotação 42.0° com interpolação
"Vizinhos mais próximos"

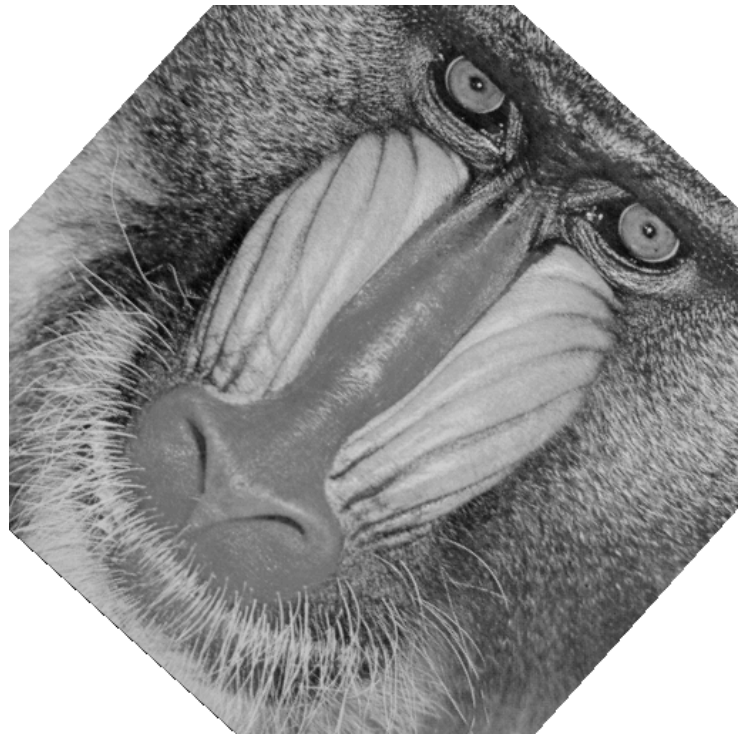


Imagem 2: rotação 42.0° com interpolação
bilinear

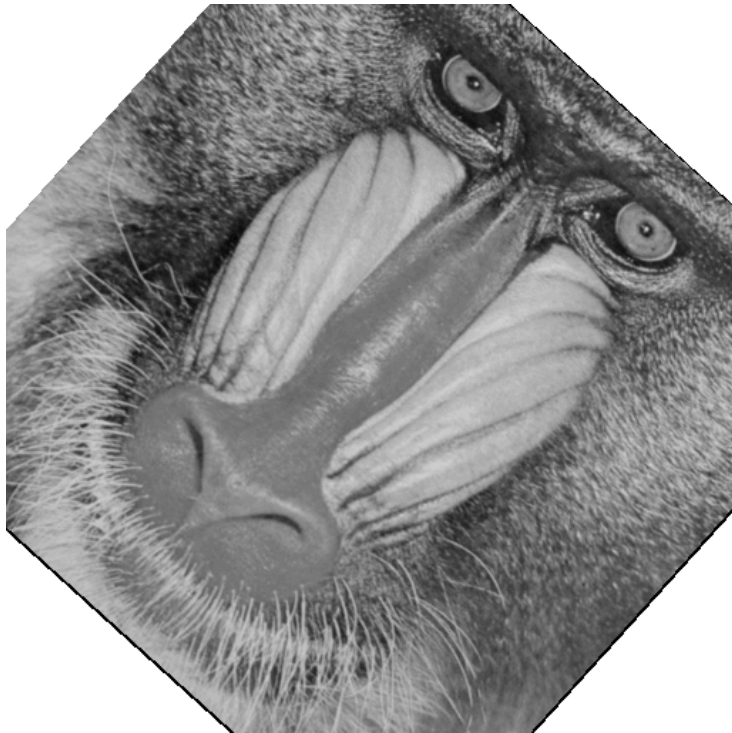


Imagem 3: rotação 42.0° com interpolação bicúbica

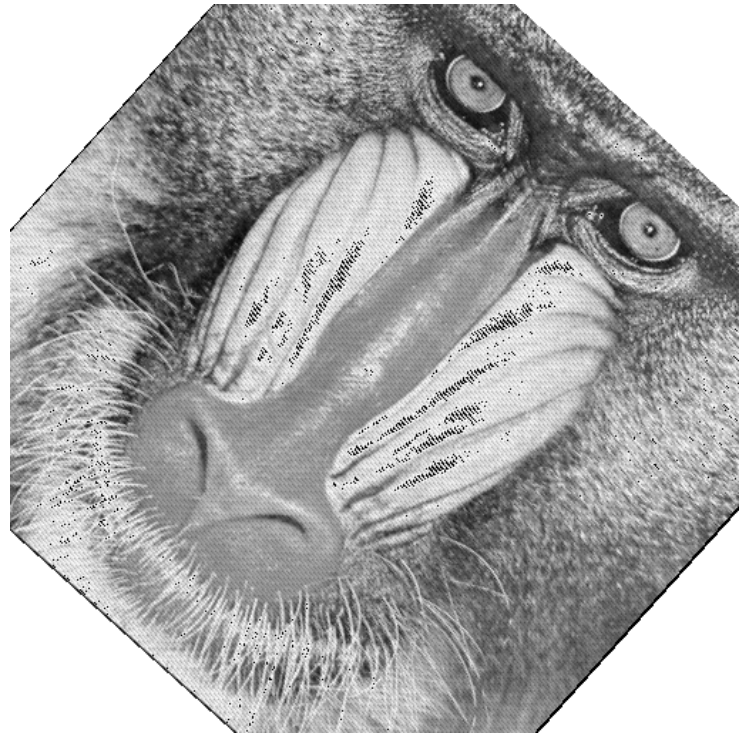


Imagem 4: rotação 42.0° com interpolação lagrangeana

Analizando as imagens, nota-se alta similaridade entre as imagens 1, 2 e 3, onde percebe-se êxito na rotação e na manutenção das informações. No entanto, o mesmo não é demonstrado na Imagem 4, a qual apresenta considerável perda de informações, embora possua a correta rotação. Considerou-se diversas causas para tal ruído nas imagens e acredita-se que pode estar na soma dos valores do polinômio. Não obstante, não foi possível comprovar a origem do erro com os testes realizados.

Agora, analisa-se as imagens que passaram por um processo de escala de aproximação.



Imagem 5: escala de 2.0x com interpolação
"Vizinhos mais próximos"

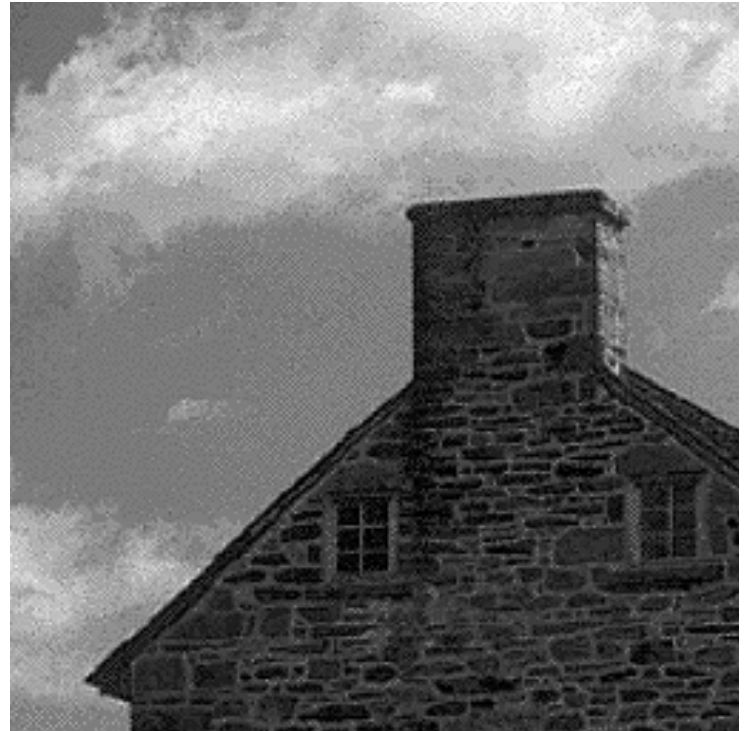


Imagem 6: escala de 2.0x com interpolação
bilinear



Imagem 7: escala de 2.0x com interpolação
bicúbica

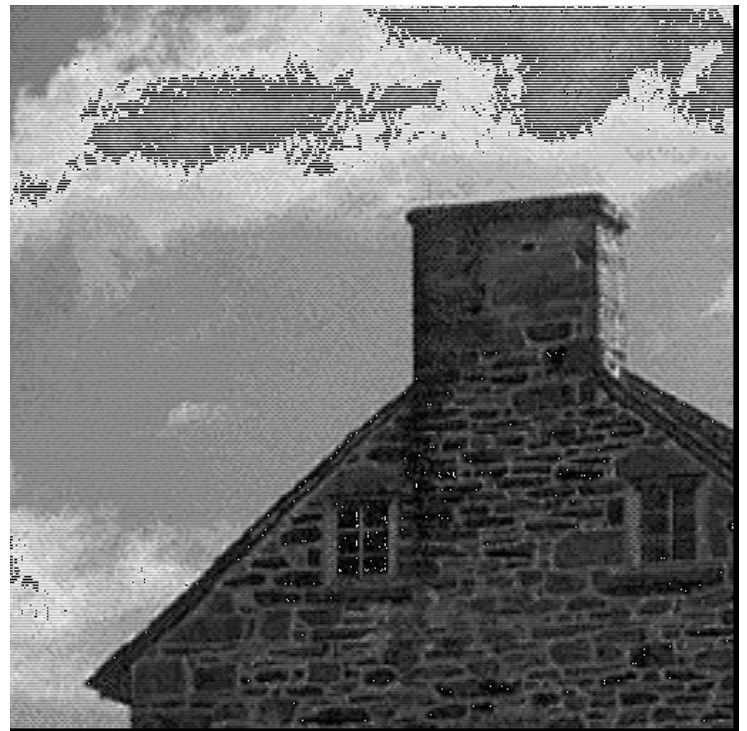


Imagem 8: escala de 2.0x com interpolação
lagrangeana

As quatro imagens apresentam diferenças sutis em relação ao resultado da interpolação. Nota-se que a Imagem 5 apresenta padrões quadriculados, enquanto a Imagem 6

apresenta granulosidade dos *pixels*, ao passo que, a Imagem 7 apresenta uma suavização em relação a imagem original.

E, novamente, percebe-se a presença de ruídos quando aplicada a interpolação lagrangeana, a qual desta vez contém “borrões” e padrões horizontais sobre a imagem.

Por fim, analisa-se o último processo, o redimensionamento da imagem. Como pode ser observado abaixo, onde aplicou-se a diminuição para dimensões 128x128.

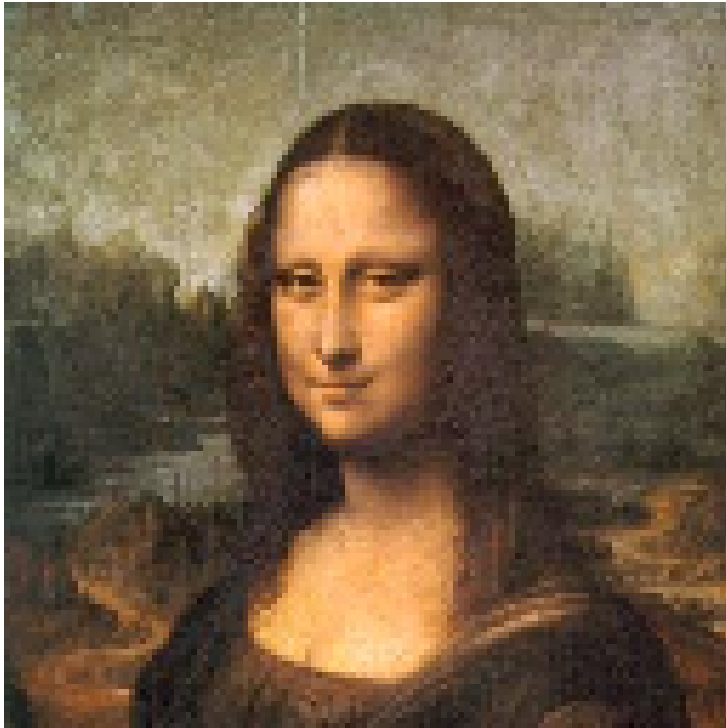


Imagem 9: redimensionamento de escala equivalente a 0.5x, com “Vizinhos mais próximos”



Imagem 10: redimensionamento de escala equivalente a 0.5x, com bilinear



Imagem 11: redimensionamento de escala equivalente a 0.5x, com bicúbica



Imagem 12: redimensionamento de escala equivalente a 0.5x, com lagrangeana

Nota-se que as imagens 9 e 10 apresentam uma “quadriculação” dos pixels da imagem original, enquanto a Imagem 11, consegue suavizar os pixels. Ademais, é possível verificar que com uma imagem colorida não há presença de ruídos ao aplicar o método lagrangeano, o que pode estar relacionado às iterações com banda de cor.

5 Bibliografia

- https://docs.opencv.org/4.x/d4/d73/tutorial_py_contours_begin.html
- <https://learnopencv.com/find-center-of-blob-centroid-using-opencv-cpp-python/>
- https://docs.opencv.org/4.x/dd/d49/tutorial_py_contour_features.html
- https://docs.opencv.org/4.x/d1/d32/tutorial_py_contour_properties.html
- <https://pt.stackoverflow.com/questions/172854/como-usar-o-sys-arg-para-entrada-de-dados>
- <https://docs.python.org/3/library/argparse.html>
- https://docs.opencv.org/4.x/d4/d61/tutorial_warp_affine.html